

Yapr: System Design & Implementation Plan

Oct 9, 2026 · @Asad

Yapr is a Twitter-style social app where every post is a Yap, built on React, Supabase and a Node backend, and shipped in three phases so v1 works end to end before any advanced piece is added.

Tech stack

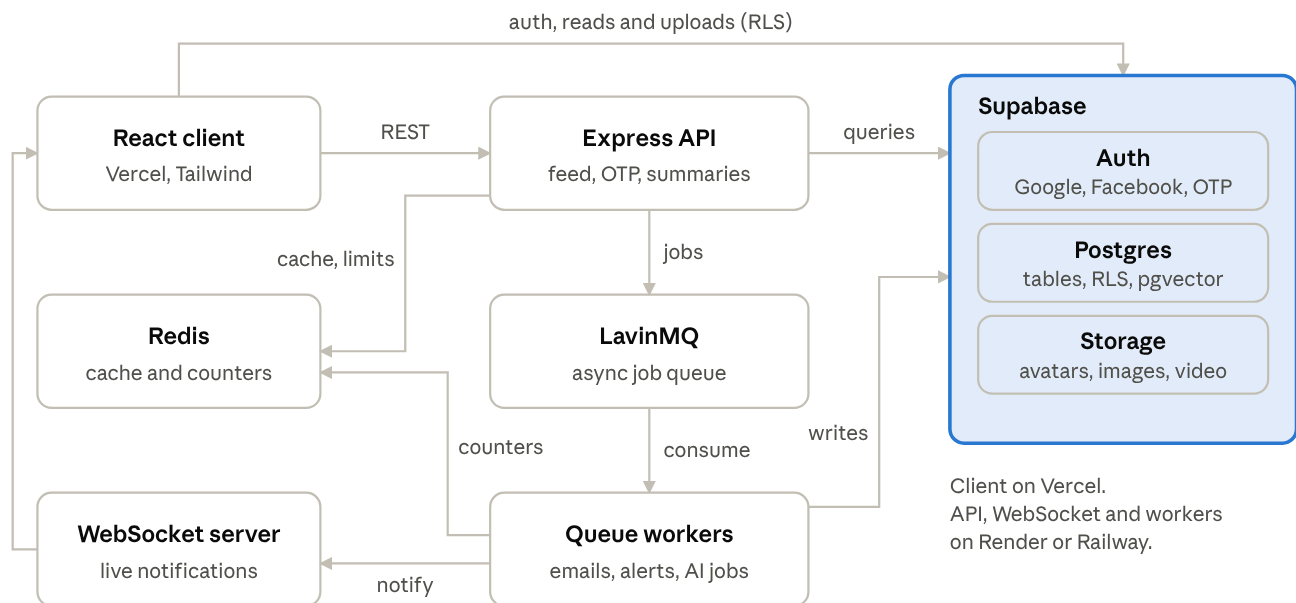
Supabase replaces MongoDB, so this is a PERN-style stack (Postgres, Express, React, Node) rather than classic MERN.

Layer	Choice	Job
Frontend	React + TypeScript, Tailwind CSS	Feed, profiles, composer, notifications UI
Backend API	Node + Express (TypeScript)	Custom logic: feed ranking, OTP, summaries, WebSocket server
Database	Supabase Postgres	Users, yaps, follows, likes, replies, reyaps, notifications
Auth	Supabase Auth (Google, Facebook OAuth)	Login, sessions, JWTs used by Row Level Security
File storage	Supabase Storage buckets	Avatars, yap images, yap videos
Cache	Redis (Upstash or self-hosted)	Feed cache, trending counters, rate limits, bloom filter
Message queue	LavinMQ (CloudAMQP in production)	Async jobs: emails, notifications, summaries, embeddings
Real-time	WebSockets (Socket.io)	Live notifications
Email	Own SMTP service	OTP emails for non-Google signups
AI	Gemini Flash and Groq models	Yap summaries (check exact model names in each provider's docs)

Layer	Choice	Job
Recommendations	Postgres + pgvector, later a Python service (FastAPI, LightGBM)	Feed scoring and personalization
Hosting	Vercel (frontend); Render, Railway or Fly.io (backend, workers)	WebSockets and queue workers cannot run on Vercel serverless

Architecture

Supabase holds the data; slow work runs through the queue



architecture · 8 components, 10 connections

The React client talks to Supabase directly for auth, reads and uploads, protected by Row Level Security, and to the Express API for custom logic. The API hands slow work to LavinMQ; workers write results to Supabase and push live notifications through the WebSocket server.

Data model

Eight core tables cover v1 and v2. Keep follows, likes and reyaps in their own tables, enable Row Level Security (RLS) on every one from day one, and use cursor-based pagination on `created_at` everywhere.

Table	Key columns	Notes and indexes
<code>profiles</code>	<code>id</code> (= auth user id), <code>username</code> (unique), <code>display_name</code> , <code>bio</code> , <code>avatar_url</code> , <code>country_code</code> , <code>follower_count</code> , <code>following_count</code>	Unique index on <code>lower(username)</code> ; <code>country_code</code> from an ISO 3166 list
<code>yaps</code>	<code>id</code> , <code>author_id</code> , <code>body</code> , <code>parent_id</code> (null for top-level, set for replies), <code>media</code> (array of urls), <code>like_count</code> , <code>reply_count</code> , <code>reyap_count</code> , <code>summary</code> , <code>edited_at</code> , <code>deleted_at</code>	Index on (<code>author_id</code> , <code>created_at</code>) and (<code>created_at</code>); soft delete via <code>deleted_at</code>
<code>follows</code>	<code>follower_id</code> , <code>followee_id</code> , <code>created_at</code>	Primary key on both ids; index on <code>followee_id</code> for fan-out
<code>likes</code>	<code>user_id</code> , <code>yap_id</code> , <code>created_at</code>	Primary key on both ids; triggers update <code>like_count</code>
<code>reyaps</code>	<code>user_id</code> , <code>yap_id</code> , <code>quote_body</code> (optional), <code>created_at</code>	Primary key on both ids
<code>hashtags</code> and <code>yap_hashtags</code>	<code>tag</code> , <code>yap_id</code> , <code>country_code</code> , <code>created_at</code>	Drives regional trending; index on (<code>tag</code> , <code>created_at</code>)
<code>notifications</code>	<code>id</code> , <code>user_id</code> , <code>type</code> , <code>actor_ids</code> , <code>yap_id</code> , <code>read_at</code> , <code>created_at</code>	Stored so offline users still see them; index on (<code>user_id</code> , <code>created_at</code>)
<code>interactions</code>	<code>user_id</code> , <code>yap_id</code> , <code>type</code> (view, like, reply, skip), <code>dwel_ms</code> , <code>created_at</code>	Written in batches; feeds the recommender in later stages

Later additions: `blocks`, `mutes`, `reports`, `bookmarks`, `yap_edits` (edit history) and a `vector` column on `yaps` for embeddings (pgvector).

- Counters (likes, replies, reypas, followers) are kept current with Postgres triggers, so no request recounts rows.
- Replies are yaps with a `parent_id`, which keeps threads in one table.

Auth and accounts

Supabase Auth handles sessions and issues the JWTs that RLS reads, so every login path must end in a real Supabase session.

1. **Google and Facebook OAuth** through Supabase Auth. Both need a Terms of Service and Privacy Policy page, and Facebook login also needs Meta app review.
2. **Email signup with OTP** for users who do not use Google. v1 uses Supabase's built-in email OTP with custom SMTP; v2 swaps in your own OTP service.
3. **Own SMTP and OTP service (v2)**. The backend generates a short-lived, hashed, rate-limited code and emails it through a queue job. After verification it must create a real Supabase session (admin API or a signed custom JWT), otherwise RLS will not recognize the user.
4. **Username availability**. A `UNIQUE` index on `lower(username)` in Postgres is the source of truth. In v3 add a bloom filter in Redis (RedisBloom or an app library) as a fast pre-check; it can say "taken" wrongly but never "free" wrongly, and it cannot delete entries, so rebuild it periodically or use a counting bloom filter.
5. **Country** is chosen from a dropdown backed by a static ISO 3166 list and saved as `country_code` on the profile.
6. **Account basics**: password reset, email verification, session list and logout, account deletion, and optional private accounts.

Feed and recommendation system

The feed starts as a simple scored query and grows into a learned model only when real usage justifies it.

Stage	What it does	Build when
1. Rule-based scoring	Ranks yaps from followed accounts with the formula below, computed in Postgres	v1, with the core app
2. Collaborative filtering	Recommends what similar users liked, using the <code>interactions</code> table	Only with real users

Stage	What it does	Build when
3. Content embeddings	Multilingual embeddings (Urdu, Roman Urdu, English) in pgvector, matched against a per-user taste vector	v3, after a few hundred yaps
4. Learned ranking	LightGBM predicts like, reply or skip; a scheduled Python service caches each user's ranked feed	Only with real users

Stage 1 formula:

```
score = (likes \times 1 + replies \times 3 + reyaps \times 2) \times
recency\_decay + author\_affinity + topic\_match
```

- **Recency decay** fades a yap's score over hours; **author affinity** rises with how often you like or reply to that author; **topic match** rises for hashtags you engage with.
- **Cold start:** new users see trending and regional yaps, and pick interests at signup.
- **Diversity mix:** reserve part of the feed for content outside the user's usual topics, and down-rank heavily reported yaps so outrage does not win.
- **User control:** a "why you're seeing this" label on suggested yaps, plus feed sliders (for example more from people I follow, less viral). Chronological order is the default option.
- **Logging:** batch view, like, reply and skip events instead of writing one row per event.

Notifications

Notifications are created by queue workers, stored in Postgres, and pushed live over WebSockets, so users who were offline still see them later.

1. A user posts a yap; the API saves it and publishes a `yap.created` job to LavinMQ.
2. A worker looks up followers and writes notification rows in batches.
3. The WebSocket server pushes each new notification to followers who are online. Sockets authenticate with the Supabase JWT, and a Redis adapter lets several servers share rooms.
4. The client shows an unread count from the `notifications` table, not from the socket alone.

Hybrid fan-out for big accounts. Fan out on write for accounts with a modest follower count. For high-follower accounts, skip per-follower writes and aggregate or pull on read, producing lines like "Ali and 40 others posted". Pick the follower threshold (for example 10,000) once you have real numbers.

Reliable workers. Every job needs retries, a dead-letter queue and an idempotency key, so a retry never sends two notifications or two emails.

Trending, search, media and AI summaries

Trending by region. Extract hashtags when a yap is saved and store them with the author's `country_code`. Count recent usage per country in Redis sorted sets with time buckets (last 1 hour and last 24 hours) so old topics fade out. Each country feed shows its top hashtags with counts.

Search. Use Postgres full-text search plus `pg_trgm` for users, yaps and hashtags, and add Roman Urdu spelling variants as aliases so "khabar" and "khabr" match.

Media. Three Supabase Storage buckets: `avatars`, `yap-images` and `yap-videos`.

- Set size and file-type limits, compress images on upload, and generate thumbnails.
- Cap video length and size in v1, because transcoding is expensive.
- Protect buckets with storage policies so users can only write their own files.

AI summarizer (v3). Summaries run through the queue, never inline in the request.

- Trigger only for long yaps or long threads, and save the result in `yaps.summary` so it is generated once.
- Apply per-user limits, and fall back between Gemini and Groq when one fails or rate-limits.
- Confirm the exact model names in each provider's documentation before coding, since they change often.

Security, abuse protection and operations

The biggest security rule: RLS on every table with explicit policies, because a table without RLS is readable by anyone holding the public key.

Safety and abuse

- Report, block and mute features, plus soft delete on yaps

- Rate limits on signup, OTP requests, posting and likes (Redis counters)
- Spam and bot protection: email verification, a CAPTCHA on signup, and throttling of new accounts
- Yap rules: character limit, a short edit window with saved edit history

Operations

- Error tracking (Sentry), structured logs and a health-check endpoint
- Secrets in environment variables, never in the repo; separate staging and production projects
- CI/CD with basic tests for auth, feed ranking and notifications
- Terms of Service and Privacy Policy pages, required for OAuth review and to disclose what behavior is tracked
- Backups: enable Supabase's database backups before real users arrive

Build roadmap

Finish each phase completely before starting the next. A working v1 matters more than ten half-built systems.

Phase	Goal	What ships
v1: core app	A usable Twitter-style app	Supabase Auth (email OTP, Google, Facebook), profiles with country dropdown, yaps with images and video, replies, likes, reyaps, follow and unfollow, Stage 1 feed, RLS and indexes, basic report and block, deployment
v2: real-time and scale	Make it feel alive and handle growth	Express backend, Redis caching, LavinMQ workers, WebSocket notifications with fan-out and aggregation, hashtags and regional trending, search, own SMTP OTP service, rate limiting

Phase	Goal	What ships
v3: intelligence	Make it smart	AI summarizer (Gemini and Groq), pgvector embeddings (Stage 3), bloom filter for usernames, interaction logging, feed sliders and "why you're seeing this"
Later	Grow with real users	Stage 2 and Stage 4 recommenders, DMs, and the ideas in the next section

Suggested order inside v1: auth, then profiles, then yaps and media, then follows and the feed, then likes, replies and reyaps.

Features for later

Start with tagged replies and Roman Urdu support after v2, since they make Yapr feel different from Twitter and Reddit and fit the Supabase setup.

Feature	What it is	Priority
Direct messages	Private one-to-one chat, built on Supabase Realtime or your WebSocket server	High
Tagged replies with upvotes	Replier labels a reply Agree, Disagree, Question or Source; replies are upvoted so the best rise	High
Roman Urdu and Urdu support	RTL layout, mixed-script posts, Roman Urdu spelling-aware search	High
Bookmarks and web push	Save yaps; push notifications when the app is closed	Medium
Stage 2 and Stage 4 recommenders	Collaborative filtering and a LightGBM ranker	Medium, once there is real usage
Voice yaps	30 to 60 second audio clips	Medium
Branch view and thread summary	Collapsible reply tree and a TL;DR at the top of long threads	Medium

Feature	What it is	Priority
Spoiler mute	Mute a topic until a date, then it returns automatically	Low
Predictions	Timestamped predictions resolved later, with a "Called it" record	Low
Close circles and scheduled yaps	Post to a chosen group of followers; schedule posts	Low
City communities and topic reputation	Lahore, Karachi and Islamabad feeds; karma per subject	Low

Decisions to confirm

These keep your design but cut moving parts in the first versions. Each is optional; the plan above works either way.

- **LavinMQ or Redis queues.** Redis with BullMQ can handle both caching and jobs at your scale, so LavinMQ could wait. If you keep LavinMQ, you run two pieces of infrastructure for jobs and caching.
- **Custom OTP now or later.** The plan builds it in v2; Supabase's built-in email OTP is enough for v1.
- **Bloom filter timing.** A Postgres unique index is fast enough until you have millions of users, so the plan puts it in v3 as a learning feature.
- **Two LLM providers.** One provider is enough at first; add the second as a fallback in v3.
- **Express backend in v1.** React talking directly to Supabase with RLS is the fastest v1; the backend becomes necessary in v2 for WebSockets, queues and custom logic.
- **Fan-out threshold.** The follower count where you switch from fan-out on write to aggregation needs real data to set.